In-Home Assistive Robot

High-Level System Design

Design D0

Gavin Lamphier	Industrial Design
Liam Patel	Mechanical Engineering
Austin Heiss	Computer Science

October 2, 2026

Goal. Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.

AI was used to format this document in LaTeX.

1 Title, goal statement, and conventions

Goal: Our robot lets an older or disabled adult at home take video calls from a family caregiver and, if they want, have the robot follow them from room to room during the call. The robot stops following as soon as it loses track of them or a safety check fails.

The basic input to the system is a caregiver's call request, the resident's choices, and readings from the robot's sensors. The basic output is a two-way video call that the resident has accepted, a clear display of the call and privacy status, and robot movement that either follows the resident or stops.

We are designing and building the robot's physical platform ourselves (the chassis, mounts, enclosure, and mechanisms) and fitting it with purchased motors, sensors, electronics, and a screen. For this first design we are focusing on calls and on following the resident when they ask, in a controlled, single-floor indoor space. We have also talked about item delivery, reminders, an adjustable screen, and help detection, but have not decided on those yet.

In each diagram, blue rounded boxes are parts we build and gray dashed boxes are outside services we rely on. Stick figures are people, and dashed blue outlines show where the software runs. Arrows point in the direction information or control moves, and the I1-I16 labels match the rows of the interface table. The red line is the planned direct wire for the Stop and Privacy buttons, and the small open-ended boxes are places where information is stored.

Every part traces back to at least one of our five user stories. Keeping a call going while the resident moves (US-01) uses the whole system. The resident controls and the call manager make sure nothing starts without the resident's consent (US-02). The call server and caregiver app tell the caregiver what happened to a call (US-03). Person tracking, motion control, and the platform keep the robot from following the wrong person (US-04). The resident controls, and where we place them, make the robot usable from a seat (US-05).

2 Block diagram (the D0 diagram)

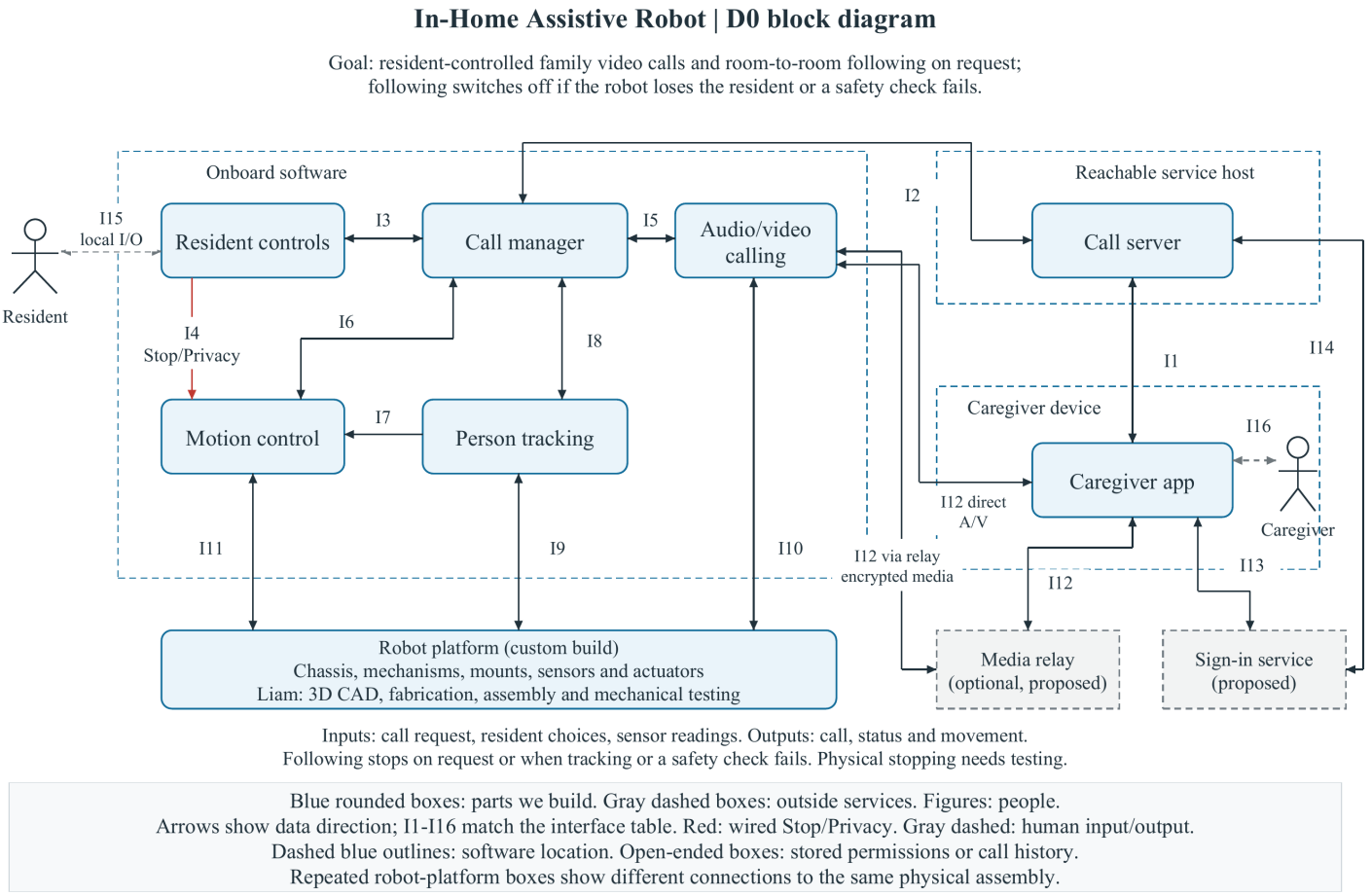


Figure 1. System block diagram

The robot's software runs on the platform we are building, while the call server and caregiver app run separately. We plan to use an outside sign-in service and, when a direct connection fails, a media relay, but we have not picked providers for either yet.

3 Component responsibility table

Liam is in charge of the mechanical build: 3D modeling, chassis and mechanism design, fabrication, assembly, and mechanical testing. He will also plan test fixtures and check fit, strength, clearances, and low-speed driving. Gavin is in charge of the robot's form, where the screen and controls go, and the experience for both the resident and the caregiver. Austin is in charge of the software, communication, sensing, and control code, and works with Liam where the mechanical and electrical parts meet.

Table 1. Responsibilities for the eight parts of the system

Component and responsibility	Interfaces	Owner and stories
Resident controls: Gives the resident the screen and buttons for calls, privacy, and following.	In: I3, I15. Out: I3, I4, I15	Gavin Lamphier. US-01, US-02, US-04, US-05
Call manager: Decides what the call and the robot are allowed to do, based on the resident's choices.	In: I2, I3, I5, I6, I8. Out: I2, I3, I5, I6, I8	Austin Heiss. US-01, US-02, US-03, US-04, US-05
Audio/video calling: Sends and receives live audio and video once the resident accepts a call.	In: I5, I10, I12. Out: I5, I10, I12	Austin Heiss. US-01, US-02, US-03, US-05
Person tracking: Reports where the selected resident is, or that it can't find them.	In: I8, I9. Out: I7, I8, I9	Austin Heiss. US-01, US-02, US-04, US-05
Motion control: Turns the resident's position into movement commands, requesting a stop whenever it should.	In: I4, I6, I7, I11. Out: I6, I11	Liam Patel. US-01, US-04, US-05
Robot platform: Is the physical robot that carries the sensors, screen, speaker, and motors.	In: I9, I10, I11. Out: I9, I10, I11	Liam Patel. US-01, US-04, US-05
Call server: Keeps track of each call attempt so the caregiver and the robot agree on its status.	In: I1, I2, I14. Out: I1, I2, I14	Austin Heiss. US-01, US-02, US-03
Caregiver app: Lets the caregiver start or end a call and see what happened to it.	In: I1, I12, I13, I16. Out: I1, I12, I13, I16	Gavin Lamphier. US-01, US-03

4 Interface specification table

Each row lists the main fields and their types, the data format, how the data is sent, and which part handles a failure. We plan for the parts on the robot to talk through function calls inside one program; we still need to choose the physical connections and the outside service providers. A call's status is one of ringing, connected, declined, no answer, offline, ended, or failed.

Table 2. Interface specifications

Connection	Specification
I1 caregiver app <-> call server	**Inputs and outputs.** Request: robot_id and access_token (text), action (start or end). Response: call_id (text), status (call status), last_seen (timestamp or none).
	Data format. JSON
	Protocol. HTTPS requests; secure WebSocket status updates
	Error behavior and handler. The call server rejects bad requests, and the app tells the caregiver when it can't connect.

Table 2. Interface specifications

Connection	Specification
I2 call server <-> call manager	**Inputs and outputs.** Invitation: call_id and caller_name (text). Response: call_id (text), status (call status); robot_online (boolean). **Data format.** JSON **Protocol.** Secure WebSocket; robot sends status each second **Error behavior and handler.** The server reports the robot offline after 5 seconds without an update; if the connection drops during a call, the call manager ends the call.
I3 resident controls <-> call manager	**Inputs and outputs.** Input: action (accept, decline, end, follow, stop, or privacy), target_id (text or none). Output: caller_name (text), call_status and follow_status (status). **Data format.** Typed in-memory records **Protocol.** Proposed in-process request and status calls **Error behavior and handler.** The call manager ignores actions that don't make sense at that moment, and the controls show an error.
I4 resident controls -> motion control	**Inputs and outputs.** Input: stop and privacy (on/off). Status returns on I6 and I3. **Data format.** Digital signal **Protocol.** Proposed direct wired contacts **Error behavior and handler.** Motion control requests a stop when Stop or Privacy is pressed, or if the wire is broken.
I5 call manager <-> audio/video calling	**Inputs and outputs.** Input: capture_allowed (boolean). Output: media_status (connecting, connected, or failed). **Data format.** Typed in-memory records **Protocol.** Proposed in-process permission and status calls **Error behavior and handler.** Audio/video calling stops capture if permission is lost; the call manager reports failure.
I6 call manager <-> motion control	**Inputs and outputs.** Input: follow_requested (boolean). Output: motion_status (following, stopping, stopped, or fault). **Data format.** Typed in-memory records **Protocol.** Proposed in-process request and status calls **Error behavior and handler.** Motion control requests a stop if following is no longer allowed or requested, and the call manager shows the new status.
I7 person tracking -> motion control	**Inputs and outputs.** Output: target_id (text), range_m and bearing_deg (numbers or none), tracking_status (valid or unavailable). **Data format.** Typed in-memory record **Protocol.** Proposed in-process position update **Error behavior and handler.** Motion control requests a stop if tracking becomes unavailable, then waits until the resident asks it to follow again.

Table 2. Interface specifications

Connection	Specification
I8 call manager <-> person tracking	**Inputs and outputs.** Input: tracking_allowed (boolean), target_id (text). Output: tracking_status (valid or unavailable). **Data format.** Typed in-memory records **Protocol.** Proposed in-process permission and status calls **Error behavior and handler.** If tracking is no longer allowed, person tracking stops and reports itself as unavailable.
I9 robot platform <-> person tracking	**Inputs and outputs.** Input: capture_requested (boolean). Output: image (image frame), distance_m (numbers), sensor_status (available or unavailable). **Data format.** Image and numeric sensor data **Protocol.** Local sensor-driver calls; physical connections to be selected **Error behavior and handler.** If the sensors aren't working, person tracking reports them as unavailable and doesn't give a position.
I10 robot platform <-> audio/video calling	**Inputs and outputs.** Input: camera_frame (image), mic_audio (audio). Output: display_frame (image), speaker_audio (audio), capture_requested (boolean). **Data format.** Image and audio buffers; typed control record **Protocol.** Local camera, audio, and display driver calls; physical connections to be selected **Error behavior and handler.** If the camera, microphone, speaker, or display fails, audio/video calling reports it and ends the call.
I11 motion control <-> robot platform	**Inputs and outputs.** Output: speed_m_s and turn_rate_deg_s (numbers). Input: measured_speed_m_s (number), path_clear (boolean), platform_status (available or fault). **Data format.** Numeric control and sensor records **Protocol.** Local motor and sensor driver calls; physical connections to be selected **Error behavior and handler.** Motion control requests a stop if sensor feedback stops arriving. We still need to test how well the robot actually stops.
I12 audio/video calling <-> caregiver app; optional media relay	**Inputs and outputs.** Both directions: video (video stream), audio (audio stream). **Data format.** Encoded audio/video packets **Protocol.** Proposed encrypted WebRTC, directly or through a relay **Error behavior and handler.** Audio/video calling reports when the connection is lost, and the caregiver app shows that the call failed.
I13 caregiver app <-> sign-in service	**Inputs and outputs.** Input: sign_in (credential text). Output: authenticated (boolean), access_token (text). **Data format.** Sign-in form and JSON response **Protocol.** HTTPS **Error behavior and handler.** The caregiver app shows that sign-in failed and won't place calls until the caregiver signs in.

Table 2. Interface specifications

Connection	Specification
I14 call server <-> sign-in service	**Inputs and outputs.** Input: access_token (text). Output: valid (boolean), caregiver_id (text). **Data format.** JSON **Protocol.** HTTPS verification request **Error behavior and handler.** If the call server can't verify a request, it rejects it and reports a service error.
I15 resident <-> resident controls	**Inputs and outputs.** Input: action (touch/button choice). Output: caller_name (text), privacy, following, camera, and microphone (on, off, or unknown), alert (sound). **Data format.** Touch/button events; text, indicators, and sound **Protocol.** Local button and screen interaction **Error behavior and handler.** Resident controls show an error if an action cannot be completed.
I16 caregiver <-> caregiver app	**Inputs and outputs.** Input: action (call or end). Output: call_status (call status), video (video stream), audio (audio stream). **Data format.** Touch/click events; text and audio/video **Protocol.** Phone or browser interaction **Error behavior and handler.** The caregiver app displays an unavailable or failed status when a call cannot connect.

For example, a caregiver starting a call on I1 would send the robot's ID, their sign-in token, and the action:

```
{"robot_id": "home-robot-01", "access_token": "example", "action": "start"}
```

5 Data-flow diagram

In the call flow, the caregiver's request goes through the call server to the robot. The resident sees who is calling and decides whether to answer. Once they accept, live audio and video go between the robot and the caregiver, and the call status comes back through the server. In this first design, the camera and microphone are only used during a call the resident has accepted.

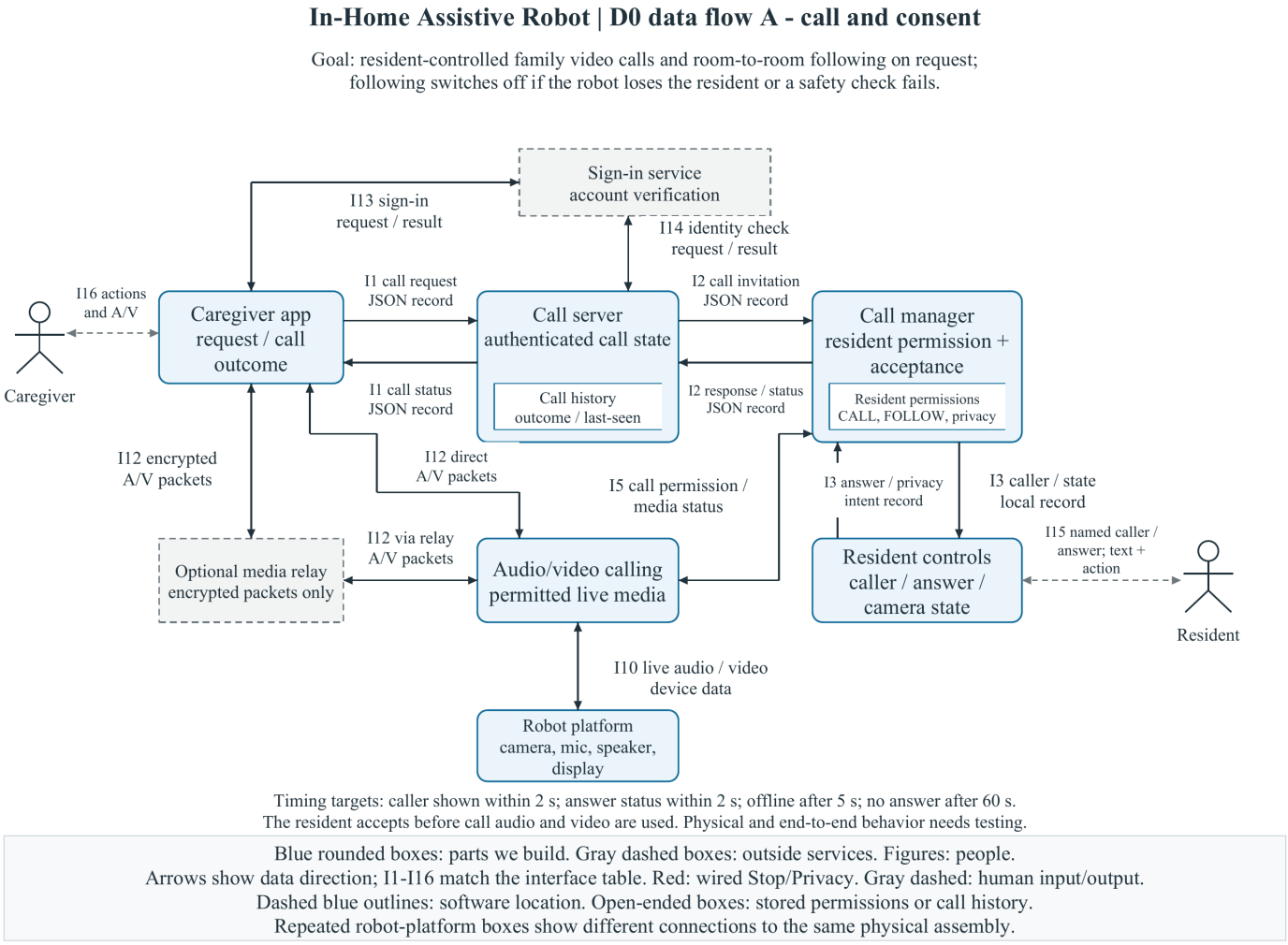


Figure 2. Call and consent data flow

In the following flow, the resident asks the robot to follow them. The robot's sensor readings are turned into the selected person's position, and motion control uses that position along with the robot's safety checks to decide how to move. If the robot loses the person, the resident turns following off, or a safety check fails, motion control requests a stop. After losing the person, the robot won't start following again until the resident asks, but the call can keep going while the robot is stopped. We still have to test how well the robot actually stops once it is built.

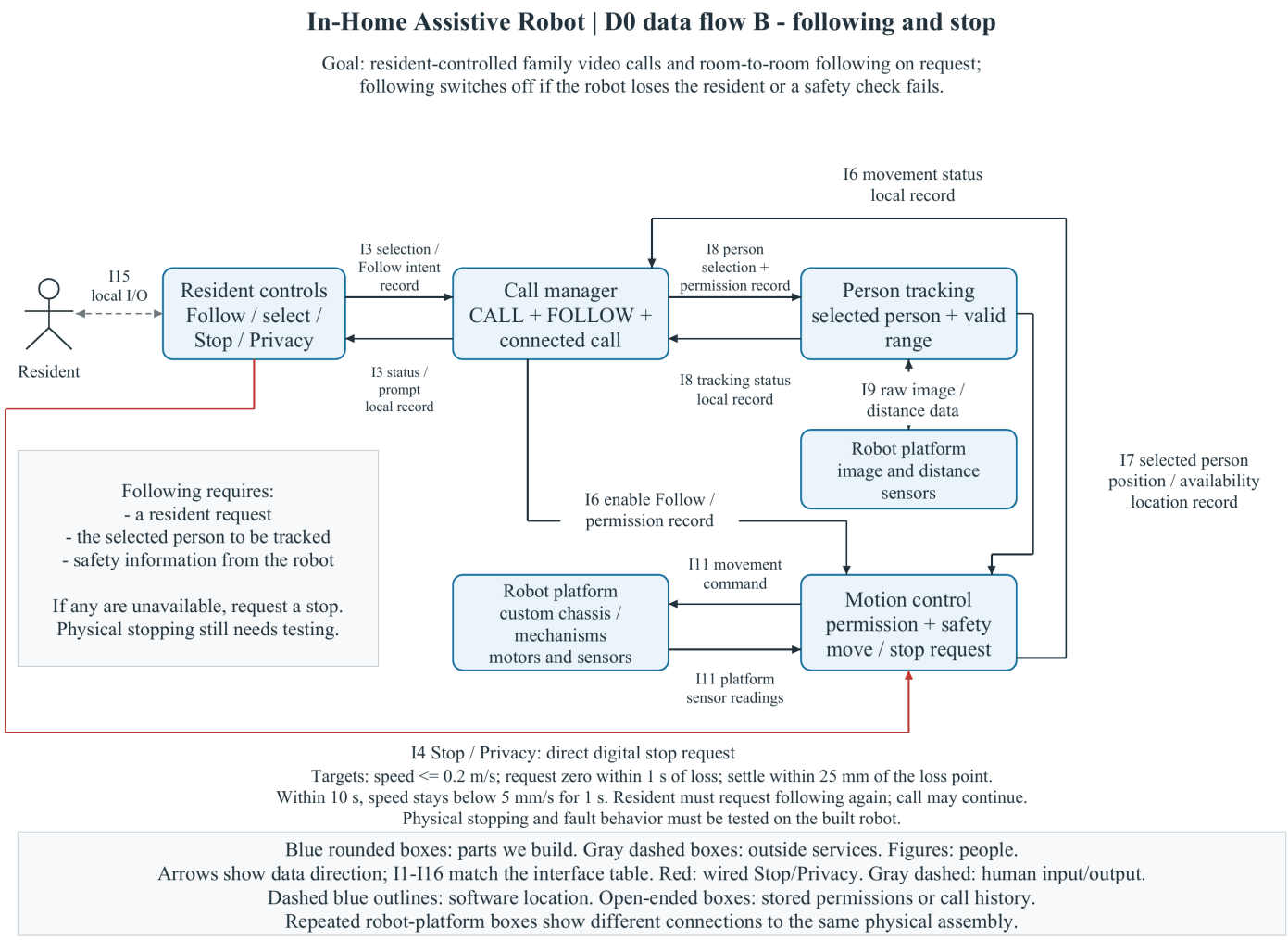


Figure 3. Following and stop data flow

Our user stories set a maximum following speed of 0.2 m/s and require the movement request to drop to zero within 1 second of losing the person. After that, the robot has to settle within 25 mm of where it lost the resident and stay below 5 mm/s for a full second, all within 10 seconds. These are targets for the design; we haven't shown them on the physical robot yet.

6 Architecture pattern and justification

We use a **client-server** pattern for communication: the caregiver app and the robot both connect to a call server that keeps track of each call's status. On the robot, a **layered** design separates the resident controls, the call manager, and the services underneath that handle communication and movement. Following works as a **pipeline** that goes from sensor readings to a position estimate to a movement command. Finally, the motion hardware follows an **embedded sensor-actuator** pattern so that movement control stays on the robot.

Fit to the problem. Our project combines remote communication with a physical robot, so we need a server for the communication side and local control for the robot side. These four patterns cover both.

Team skills. The split matches what each of us is good at: Gavin leads interaction design, Liam leads the mechanical build and testing, and Austin leads software and sensing.

Performance and timing. The robot has to react to a lost person or a Stop press even if the internet goes down, so movement and stopping are handled on the robot itself. We still need to measure the timing and stopping behavior on the built robot.

Scalability. One call server is enough for our first robot and caregiver, and it could support more users later if needed. We decided against **microservices** for the first build, because running several separate services would add a lot of setup and maintenance without a real benefit at our scale.

Hardware constraints. The robot's size, battery, computing power, and cost all limit our hardware choices, which are still open. Our build and hosting budgets aren't approved yet. Before choosing a provider, we will compare a parts and fabrication estimate (with some contingency) against our available funds, and look at ongoing hosting and support costs.

Privacy and accessibility also shaped the design. The resident decides when to accept a call, when the robot follows, and when privacy mode is on, and the controls have to be usable from a seated position. The first version doesn't record any audio or video. During design and physical testing, we will look at where the camera and screen go, how the privacy controls work, and whether the robot creates a tripping hazard. If we add help detection later, we still need to work out how it could respect the resident's privacy.

7 Decision log

1. **Build the platform ourselves.** We considered buying a complete commercial base, but that would leave out most of our mechanical design work. Building a custom chassis lets Liam model, fabricate, assemble, and test the robot around the components we pick.

2. **Start with calling and requested following.** We also considered item delivery, reminders, and help detection. Calling and following give us one clear scenario for building out the communication and mobility platform, and we can still add the other features later.

3. **Use a call server.** We considered connecting the app and robot directly (peer-to-peer only), but then nothing could tell the caregiver the robot is offline when it drops off the network. A small server gives the app and the robot one place to exchange call requests and status.

4. **Keep movement control on the robot.** Sending movement decisions through a remote service would make the robot depend on the network to move or stop. Keeping control local means it can react quickly when it loses the person or someone presses Stop.

5. **Require resident choice for calls and following.** We considered starting calls or following automatically, but that could surprise the resident and raise privacy concerns. Instead, the resident has to accept each call and separately ask the robot to follow.

6. **Use an initial fixed screen and sensor mounts.** An extending screen or a moving sensor might give a better view, but it adds mechanical and integration work. A fixed setup is easier to build and test first, and we can decide later whether an adjustable screen is worth it.